```
N-Queens Problem
Solved using: Backtracking + Branch and Bound
CSP: Constraint Satisfaction Problem
"""

import matplotlib.pyplot as plt
import numpy as np

# ─────────────────────────────────────────────────────────────
# BRANCH AND BOUND SETS (this is what makes
# it faster than plain backtracking)
# ─────────────────────────────────────────────────────────────
cols_used       = set()   # which columns are already attacked
diag1_used      = set()   # which \ diagonals are attacked  (row - col)
diag2_used      = set()   # which / diagonals are attacked  (row + col)

solutions       = []      # store all valid solutions
call_count      = [0]     # count how many times we recurse

def is_safe(row, col):
    """
    Branch & Bound check:
    Instead of scanning the whole board, we check 3 sets in O(1).
    If any set already contains this column/diagonal → pruned (bounded).
    """
    return (col        not in cols_used  and
            (row - col) not in diag1_used and
            (row + col) not in diag2_used)

def solve(row, n, board):
```

```
    """
    Backtracking function.
    row  : current row we are placing a queen on
    n    : total queens / board size
    board: list where board[row] = column of queen in that row
    """
    call_count[0] += 1

    # ── BASE CASE ──────────────────────────────────────────
    # All n rows filled → valid solution found
    if row == n:
        solutions.append(board[:])   # save a copy
        return

    # ── TRY EVERY COLUMN IN THIS ROW ──────────────
    for col in range(n):

        # BOUND: skip this column if it's already under attack
        if not is_safe(row, col):
            continue                 # ← pruning happens here

        # ── PLACE QUEEN ────────────────────────────────
        board[row] = col
        cols_used.add(col)
        diag1_used.add(row - col)
        diag2_used.add(row + col)

        print(f"  Placed queen at row={row}, col={col}")

        # ── RECURSE (go to next row) ──────────────
```

```
        solve(row + 1, n, board)

        # ── BACKTRACK ──────────────────────────────────
        # Remove the queen — undo the choice
        cols_used.discard(col)
        diag1_used.discard(row - col)
        diag2_used.discard(row + col)
        board[row] = -1

        print(f"  Backtrack from row={row}, col={col}")

# ──────────────────────────────────────────────────────────
# VISUALIZE ONE SOLUTION
# ──────────────────────────────────────────────────────────
def visualize(solution, n, sol_number=1):
    board = np.zeros((n, n))

    # chess-board coloring
    for i in range(n):
        for j in range(n):
            if (i + j) % 2 == 0:
                board[i][j] = 0.8    # light square
            else:
                board[i][j] = 0.3    # dark square

    fig, ax = plt.subplots(figsize=(6, 6))
    ax.imshow(board, cmap="gray", vmin=0, vmax=1)

    # place queen emoji / marker
    for row in range(n):
```

```
        col = solution[row]
        ax.text(col, row, "♛", fontsize=28,
                ha="center", va="center", color="red")

    ax.set_xticks(np.arange(-.5, n, 1))
    ax.set_yticks(np.arange(-.5, n, 1))
    ax.set_xticklabels([])
    ax.set_yticklabels([])
    ax.grid(color="black", linewidth=1.5)
    ax.set_title(f"{n}-Queens  |  Solution #{sol_number}", fontsize=13)
    plt.tight_layout()
    plt.show()

# ──────────────────────────────────────────────────────────
# MAIN
# ──────────────────────────────────────────────────────────
def main():
    n = int(input("Enter N (e.g. 4, 5, 8): "))

    board = [-1] * n   # board[row] = column of queen (-1 = empty)

    print(f"\nSolving {n}-Queens with Backtracking + Branch & Bound...\n")
    solve(0, n, board)

    print(f"\n{'='*45}")
    print(f"Total solutions found : {len(solutions)}")
    print(f"Total recursive calls : {call_count[0]}")
    print(f"{'='*45}")

    if solutions:
```

```
        print("\nAll solutions (board[row] = column):")
        for i, sol in enumerate(solutions, 1):
            print(f"  Solution {i}: {sol}")

        print("\nVisualizing Solution #1 ...")
        visualize(solutions[0], n, sol_number=1)

        # uncomment below to see ALL solutions one by one
        # for i, sol in enumerate(solutions, 1):
        #     visualize(sol, n, sol_number=i)
    else:
        print("No solution exists.")

if __name__ == "__main__":
    main()
```